IoT Communications Protocols

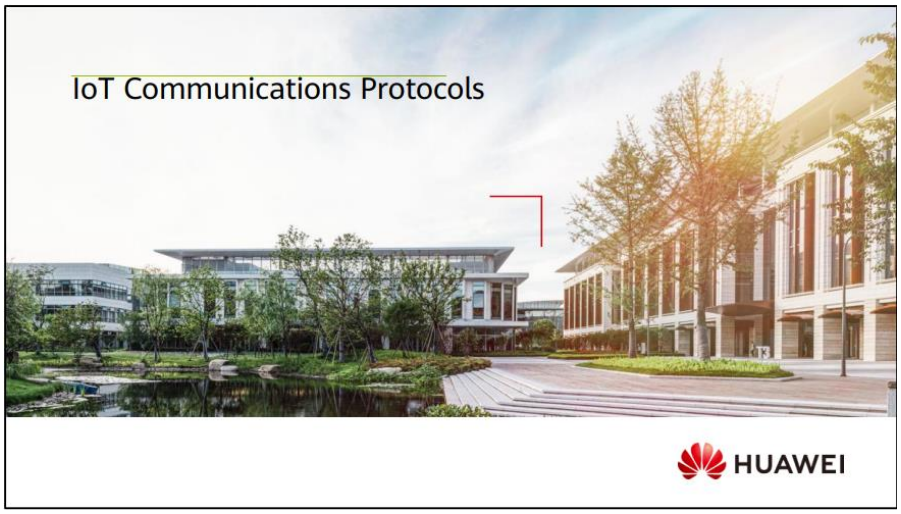
Kelompok : 8

Anggota : Muhammad Ikhsan Kamil

Muhammad Saddam A

Rezan Candrawinata

Kelas : 3NK



1. Basics of Network Communications

1.1 Communications Protocols

Communications protocols are rules and conventions that entities must follow to establish communications or provide services. A communications channel connects devices at different or same locations to form a system. In this system, devices must use the same language to share information and resources with each other. Information interactions include at least three parts **transmitters**, **transport channels**, and **receivers**.



- **A communications protocol consists of three elements**:

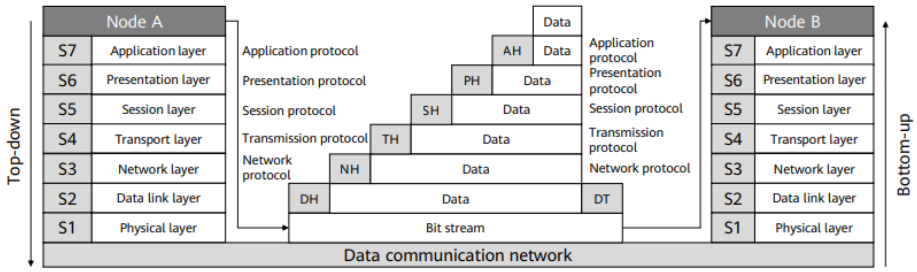
➢ Semantics specifies control information and actions required for communication entities.

➢ Syntax specifies the format and structure of data or control information exchanged between communication entities.

➢ Timing specifies the response relationship between communication entities, including speed matching and sequencing.

- **OSI Reference Model**

The open system interconnection (OSI) reference model is a standard framework proposed by the International Organization Standardization (ISO) to connect different computers around the world in a network.



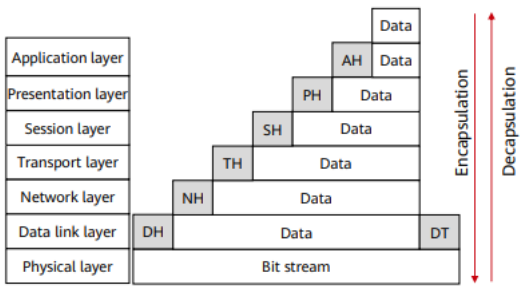
From the data above, there are several points:

- The physical layer establishes, maintains, or cuts off physical connections between two network nodes for data transmission.
- The data link layer transmits data frames between network nodes. It encapsulates bit streams at the physical layer into frames and transmits the frames to the network layer. It also breaks down frames at the network layer into bit streams and transmits the bit streams to the physical layer.
- The network layer selects the best possible route for packets or segments through the communication subnet based on the routing algorithm.
- The transport layer allows reliable data transmission between network nodes. It isolates the application layer from other data transmission layers, converts data into the format required for network transmission, checks transmission results, and corrects failed transmissions.
- The session layer is responsible for negotiation and connection between applications or processes of network nodes. It not only establishes proper connections, but also authenticates identities of both session parties.
- The presentation layer ensures that commands and data of an application can be understood by other nodes on the network. This simplifies communications and makes communications device independent.
- The application layer provides services for users and completes user tasks on the network.
- OSI data transmission process: To send data to node B, node A encapsulates data, and protocol information is added to the data at each layer. Then, data is converted into bit streams consisting of 0 and 1 and transmitted to network connection media. After node B receives the data, it breaks down the encapsulated data at each layer to complete a communication process.

○ Data Encapsulation and Decapsulation

Encapsulation is the process of adding data to a protocol packet header when it is transmitted from the top to bottom layers over the network. Decapsulation is the process of removing encapsulated information from received data and transmitting the data from the bottom to top layers based on the encapsulated information. Data is encapsulated and decapsulated:

- Inside the protocol.
- During network transmission.



1.2 Network Topologies

A network topology refers to the physical (wired) or logical (wireless) layout of various devices connected through transmission media. If two networks have the same connection structure, they have the same network topology, although the two networks may have different physical cable connections and node distances. There are different types of network topologies. Each type has its own advantages and disadvantages.

Star Topology

- A star topology is a star-shaped network topology in which each device in the network is individually connected to a central node. It is easy to implement, but is labor intensive to set up and maintain.
- Advantages:
  - Its simple structure makes it easy to manage and debug.
  - It is easy to control, add, or delete a device.
  - Centralized management facilitates service provisioning and network reconfiguration.
  - Each device directly connects to the central node, facilitating fault detection and isolation.

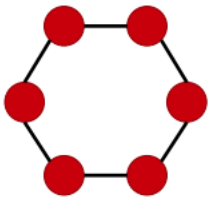


- Disadvantages:
  - The cable utilization is low. Each cable is used only by the central node and a single device.
  - The central node is prone to getting overloaded. If the central node fails, the entire network will fail, so the central node needs to be extremely reliable and includes redundant design features.
  - Installation and maintenance are expensive because a lot of cables are needed.

A star topology has the following features: simple structure, easy management, simple control, easy network construction, lower network latency, and low transmission error. Its disadvantages include high costs, low reliability, and poor resource sharing capability.

Ring Topology

- A ring topology is a type of network topology where each node is connected to two other devices on either side and transmission cables form a circular ring. Data is transmitted in one direction along the ring from one node to another.
- Advantages:
  - Its transmission rate is high.
  - Data flows only in one direction, and there is only one path between two nodes. This simplifies path selection.
  - Each node in the circular ring is bootstrapped, so the control software is simple.

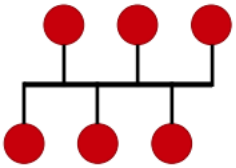


- Disadvantages:
  - Response delay: Too many nodes in the ring will slow down transmission and cause network response delay.
  - Weak scalability: A closed ring topology is less scalable than a star topology. To add or move nodes, the entire network has to shut down.
  - Difficult to maintain: If a node is faulty, the entire network will shut down.

A ring topology has the following features: Data flows only in one direction, and there is only one path between two nodes. This simplifies path selection. Each node in the circular ring is bootstrapped, so the control software is simple. If there are too many nodes in the ring, the transmission will slow down and network response will be delayed. The ring is closed, so it is difficult to expand it. A ring topology has low reliability. If a node is faulty, the entire network will shut down. It is difficult to maintain and locate faults on nodes.

Bus Topology

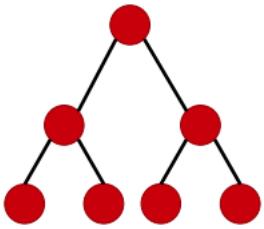
- A bus topology is a type of network topology in which nodes are directly connected to a main half-duplex line, so it is also called linear bus topology. In a bus network, each host receives all network traffic from the main line, and the transmission priority for traffic generated by each is the same.
- Advantages:
  - It is very easy to connect computers or peripherals to a linear bus.
  - It uses fewer cables than a star topology does.
  - It is suitable for small-scale networks.
  - It is inexpensive to implement.
- Disadvantages:
  - If the main cable is disconnected, the whole network fails.
  - If the entire network is interrupted, it will be difficult to locate faults.
  - Too many devices on the network will slow down transmission.



A bus topology has a simple structure and good scalability. When you want to add a node, you only need to add a branch interface to the bus to connect to the new node. You can also expand a bus when it is overloaded. A bus topology requires fewer cables, is easy to install, and uses simple and reliable devices, but it is difficult to maintain and locate node faults.

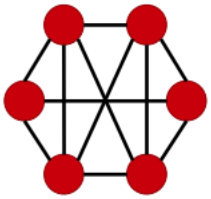
Tree Topology

- A tree topology is used for hierarchical management and control. Compared with a star network, a tree network has shorter communication lines, lower costs, and is easier to replicate, but its structure is more complex than that of a star network. A failure on any node or cable other than the leaf nodes affects all of the branches below it on the network.
- Advantages:
  - Expansion: It is easy to add new nodes and branches to a tree network.
  - Fault isolation: It is easy to isolate a branch with a faulty node or cable from the entire system.
- Disadvantages:
  - High dependency on the root node: If the root node is faulty, the entire network will be affected. This reliability issue is similar to that of a star topology.



Mesh Topology

- A mesh topology is a type of network in which information and control instructions are transmitted between network nodes through dynamic routing. In a mesh network, all nodes are connected to each other. If a node fails, data will just hop to the destination following a new route.
- Advantages:
  - Easy deployment and installation
  - High stability
  - Flexible structure
  - High bandwidth
  - It can be used outdoors.
- Disadvantages:
  - The structure is complex, expensive, and difficult to maintain.



How H

The HTTP connection between the client and server uses a one-time connection, so only one request is processed on each connection. Once the server sends a response to the request, the connection is immediately closed and re-established for the next request. This is done to avoid piling up connections on the server which can affect its performance. In addition, HTTP is also a stateless protocol, which means that the server does not store the status of completed user transactions. This helps reduce the server's memory load and allows for faster responses. HTTP is also an object-oriented protocol, which means any type of data object can be transmitted through this protocol. This protocol also supports identification of the content and size of the transmitted data as well as data compression for transmission. When a user defines a hypertext link in an HTML document, the browser will connect to the designated server using TCP/IP. HTTP supports persistent connections. In HTTP/0.9 and HTTP/1.0, the connection is closed after a request/response pair. HTTP/1.1 introduced a keep-alive mechanism that allows connections to be reused for multiple requests. Such a persistent connection significantly reduces request latency because the client does not need to renegotiate the TCP three-way handshake connection after sending the first request. Another positive effect is that the connection becomes faster over time due to the slow start mechanism of TCP.

HTTP Request

GET

The GET method requests a representation of the specified resource. Requests using GET should only retrieve data.

HEAD

The HEAD method asks for a response identical to a GET request, but without the response body.

POST

The POST method submits an entity to the specified resource, often causing a change in state or side effects on the server.

PUT

The PUT method replaces all current representations of the target resource with the request payload.

DELETE

The DELETE method deletes the specified resource.

CONNECT

The CONNECT method establishes a tunnel to the server identified by the target resource.

OPTIONS

The OPTIONS method describes the communication options for the target resource.

TRACE

The TRACE method performs a message loop-back test along the path to the target resource.

PATCH

The PATCH method applies partial modifications to a resource.

AMQP

AMQP (Advanced Message Queuing Protocol) mengikuti alur kerja tertentu di mana produsen mengirimkan pesan ke bursa, yang kemudian mendistribusikan pesan tersebut ke antrian berdasarkan aturan perutean. Pesan tersebut pada akhirnya dikirim ke konsumen yang berlangganan antrian, atau konsumen dapat mengambil pesan sesuai permintaan. Alur kerja ini dapat melibatkan beberapa produsen, bursa, antrian, dan konsumen, yang dapat berada pada perangkat yang berbeda. Produsen memiliki kemampuan untuk menentukan atribut pesan, beberapa di antaranya digunakan oleh broker dan yang lainnya hanya digunakan oleh aplikasi yang menerima pesan. Dalam situasi di mana jaringan tidak dapat diandalkan atau aplikasi konsumen gagal, AMQP menyediakan pengakuan pesan untuk mencegah hilangnya pesan. Pesan hanya akan dihapus dari antrian ketika pesan tersebut menerima pengakuan dari konsumen. Selain itu, ketika sebuah pesan tidak dapat dirutekan atau ketika sebuah ekstensi diimplementasikan oleh perantara pesan, produsen dapat memilih bagaimana menangani situasi ini dengan menggunakan parameter tertentu.

AMQP Main Core

The producer sends the message to the connected AMQP broker. Network connections between AMQP brokers and applications (including manufacturers and consumers) use TCP/IP three-way handshakes and four-way handshakes. AMQP connections are usually continuous and use TCP to ensure reliable delivery. AMQP uses authentication and provides Transport Layer Security (TLS) protection. When an application no longer needs to connect to the AMQP broker, it closes the AMQP connection instead of the TCP connection. Exchange is a connection that receives messages and routes them to the bound queue. It does not store messages. An exchange is an AMQP entity used to send messages. An exchange takes messages and routes them into zero or more queues. The routing algorithm used depends on the type of exchange and the binding rules. Exchanges can be durable or temporary. Durable exchanges can survive broker turnover. Transient exchanges do not (they must be redeclared after the broker restarts). Queues in AMQP are similar to queues in other message and task queuing systems: they store messages that will be consumed by applications. The consumer receives the messages. In AMQP, consumers can obtain messages in one of the following ways: Messaging middleware sends the message to the consumer, consumer retrieves the message. When multiple consumers join the same queue, the messages in the queue are still only consumed by one consumer (not by multiple consumers).

MQTT Overview:

- **Definition:** MQTT, or Message Queuing Telemetry Transport, is a protocol designed for devices with limited computing capabilities operating on low-bandwidth and unreliable networks. It is widely used in the Internet of Things (IoT) domain.
- **Protocol Type:** MQTT is a TCP-based lightweight messaging protocol.
- **Communication Model:** MQTT employs a publish/subscribe model, where clients publish and subscribe to messages through a broker.
- **Design Principles for IoT:**
    - Simplify functions.
    - Provide a publish/subscribe model.
    - Allow dynamic creation of topics with no operational and maintenance costs.
    - Minimize transmission volume for efficiency.

- Consider low bandwidth, high latency, and unstable networks.
- Enable control of continuous sessions.
- Suitable for clients with low computing power.
- Provide Quality of Service (QoS).
- Maintain flexible types and formats of transmitted data.

MQTT Core Components:

- **Client:**
  - Program or device using MQTT.
  - Can establish network connections, act as a publisher or subscriber, unsubscribe, and close network connections.
- **Server (Broker):**
  - Acts as a broker between publisher and subscriber clients.
  - Manages network connection requests, processes messages, handles subscriptions, and forwards messages to subscribed clients.

MQTT Network Connection & Application Message:

- **Network Connection:**
  - Provided by the underlying transport protocol.
  - Connects a client to a server, allowing ordered, lossless streams of bytes in both directions.
- **Application Message:**
  - Carries payload data, Quality of Service (QoS), properties, and topic name.

MQTT QoS (Quality of Service):

- MQTT supports three levels of QoS:
  - QoS 0: At most once.
  - QoS 1: At least once.
  - QoS 2: Exactly once.

MQTT Topic Names and Topic Filters:

- Topics are classified by names; filters can use wildcard characters.
- Wildcards include topic level separator (/), multi-level wildcard (#), single-level wildcard (+), and special wildcard ($).

MQTT Keep Alive:

- Based on TCP, MQTT provides a Keep Alive mechanism to detect half-open connections.
- Keep Alive is a two-byte integer in the CONNECT packet, indicating the maximum time interval between packet transmissions.

- If no packets are sent within the allowed interval, the server closes the connection.

MQTT Control Packet:

- Consists of a fixed header, variable header, and payload.
- Fixed header includes packet type and remaining length.

MQTT Fixed Header: Packet Type:

- Packet type is represented in the fixed header and includes various control packet types (e.g., CONNECT, PUBLISH, SUBSCRIBE).

This summary provides an overview of the key concepts and components of MQTT, including its design principles, core components, network connection, QoS, topic names, Keep Alive mechanism, and control packets.

MQTT Fixed Header: Flag:

- The first byte of the fixed header contains flags for each MQTT control packet type.
- Flags are represented in bits 3–0 of the first byte.
- Flag values are specified for each packet type, and "reserved" flags must be set to listed values.

Notable Flags:

- **DUP (Duplicate):** Used in PUBLISH packets to indicate duplicate delivery.
- **QoS (Quality of Service):** Used in PUBLISH packets to denote the QoS level.
- **RETAIN:** Used in PUBLISH packets to signify the retained message flag.

MQTT Variable Header: Packet Identifier:

- Certain packets have a variable header containing a packet identifier field.
- A packet identifier is a 2-byte integer used to identify a packet during exchanges between a client and server.
- Packet identifiers are assigned independently by the client and server, allowing concurrent message exchanges.

Usage of Packet Identifiers:

- If QoS value is 0 in a PUBLISH packet, it must not contain a packet identifier.
- New SUBSCRIBE, UNSUBSCRIBE, or PUBLISH packets (QoS > 0) must have a non-zero, currently unused packet identifier.
- After processing acknowledgment packets, packet identifiers can be reused.
- Packet identifiers for PUBLISH, SUBSCRIBE, and UNSUBSCRIBE packets form a unified set.

MQTT Variable Header: Properties:

- Found in the variable header, properties are present in most packet types except PINGREQ and PINGRESP.
- Properties consist of an identifier and a value, with the identifier specifying usage and data type.

- Example properties include payload format indicator, message expiry interval, response topic, subscription identifier, maximum topic alias, maximum QoS, and maximum packet size.

Notable Property:

- **Property Length:** Encoded as a variable byte integer, indicating the length of properties (excluding the bytes used for encoding). If no properties, length is zero.

MQTT Variable Header: PUBLISH Packet Example:

- Example illustrating the variable header structure of a PUBLISH packet.
- Includes topic name, packet identifier, and property length fields.
- Demonstrates the binary representation of bytes in the variable header.

MQTT Payload:

- Certain control packets contain a payload as the final part of the packet.
- Packets with payloads include CONNECT, PUBLISH (optional), SUBSCRIBE, SUBACK, UNSUBSCRIBE, and UNSUBACK.

This summary covers the flags in the fixed header, the packet identifier and properties in the variable header, an example of a PUBLISH packet's variable header, and the presence of payloads in specific control packets.

CoAP Overview:

- **Definition:** Constrained Application Protocol (CoAP) is a specialized Internet application protocol designed for constrained devices to communicate with the wider Internet.
- **Purpose:** CoAP is intended to be easily translatable to HTTP for Web integration while meeting IoT device communication requirements, such as multicast support, low overhead, and simplicity.
- **Compatibility:** CoAP can run on most devices supporting User Datagram Protocol (UDP).
- **Use Cases:** CoAP is designed for use between devices on constrained networks, devices on the Internet, general nodes, and devices connected over the Internet on different constrained networks.
- **Features:**
    - Based on REST with resource addresses similar to URL format.
    - Binary format, more compact than HTTP.
    - Lightweight, minimum CoAP message length is only 4 bytes.
    - Supports reliable transmission, data retransmission, and block transmission.
    - Supports IP multicast, enabling requests to be sent to multiple devices simultaneously.
    - Supports non-persistent connection communications, suitable for low-power IoT scenarios.

CoAP Logical Layering:

- CoAP uses a two-layer architecture: the messaging layer handles UDP and asynchronous interactions, and the request/response layer interacts with applications using REST request methods and response codes.

CoAP Messaging Layer: Message Types:

- Four types of messages: Confirmable (CON), Non-confirmable (NON), Acknowledgment (ACK), and Reset (RST).
- Message types and response codes determine if a message carries a request or response.

CoAP Messaging Layer: Transmission Reliability:

- Uses Confirmable (CON) messages for reliable transmission, retransmitted until the recipient sends an ACK message with the same message ID.
- Non-confirmable (NON) messages do not require acknowledgment but have message IDs for duplicate detection.

CoAP Request/Response Layer: Response Classification:

- CoAP messages carry request or response semantics, including Method Code or Response Code, respectively.
- Different types of requests receive different types of responses based on how the response is sent.

CoAP Request/Response Layer: Responses to CON Messages:

- Piggybacked response: If the requested information is immediately available, the response is carried in the ACK message.
- Separate response: If the server cannot respond immediately, an empty ACK is sent, and the response is sent in a new CON message.

CoAP Request/Response Layer: Responses to NON Messages:

- If a request is carried in a NON message, the response will be returned in a new NON message. The server may respond with a CON message.

CoAP Message Format:

- CoAP messages are encoded in a simple binary format, starting with a fixed 4-byte header, followed by a token, options, and payload.

CoAP Message Fields: Code:

- Code is in X.XX format, split into a 3-bit class and 5-bit details.
- Classes include Request (0.XX), Success response (2.XX), Client error response (4.XX), Server error response (5.XX).

CoAP Message Fields: Options:

- Options provide properties for requests and responses, specifying destination host port, proxy usage, etc.
- Options fall into critical (must be recognized) and elective (can be ignored) classes.

CoAP Request/Response Message Examples:

- Example CoAP request and response message formats with details like method, message ID, token, and options.

This summary covers the key aspects of CoAP, including its purpose, features, logical layering, message types, transmission reliability, response classification, message format, code fields, options, and practical message examples.